

Implementation of SpMV on HPC

Alex Chambers

Abstract:

Sparse Matrix Vector Multiplication (SpMV) is essential to numerous computational problems. Due to this, having efficient methods to solve this problem is extremely important. Due to a seeming gap in research on SpMV method on HPCs and me recently taking courses covering HPCs and GPU computing, I wanted to implement a GPU optimized SpMV solution on HPCs and see the performance of such an algorithm. While I am successful in implementing the algorithm and am able to do some performance analysis, due to the sheer number of variables usable in this algorithm to fine tune performance, more research on this implementation is required.

Introduction:

Sparse Matrix Vector Multiplication is a very common problem in computation, as it is used in various technologies, such as graphics processing, numerical analysis, and graph problems. Because of the wide applications of SpMV, there is currently a large amount of ongoing research in new algorithms for solving this problem, and more specifically, the optimization of such algorithms for particular hardware such as CPUs or GPUs. HPCs as a platform for solving SpMV is currently not a very investigated topic. Thus, the hope of this project was to get a better idea of how HPCs work as a platform for solving this problem. I am doing this by implementing a GPU Optimized Algorithm [1]. This algorithm is an NNZ-splitting algorithm, meaning it subdivides the matrix into same-size groups of non-zero elements and then has each device individually solve its assigned group. Additionally, this program will utilize CSR (compressed sparse row) data format to represent our matrix and vector in order to preserve space. In this project, I will implement and adjust this algorithm to function on the HPC. I will then test its runtime with a data set of size 19753 and size 79210; next, I will compare how changes in device count and thread count on the devices change performance.

Method:

This method works by first initializing MPI and OpenMP threading; next, it creates the transfer variables for MPI. Once the basic initialization is done, we first enter our device of rank 0. This is the first of our two overhead devices. Here is where we read the matrix and vector definitions (currently, because I was unsuccessful in implementing file reading, we are just defining the matrix and vector directly in the code). We then do additional overhead variable calculations, like the number of non-zero, the number of elements each device calculates, etc. Once all the overhead variable creation is done, we send to each device with a rank > 1 the necessary overhead variables, those being the total number of workgroup iterations, the number of rows and columns, the number of elements each device, and thread calculate per iteration, the row pointer array, and finally the vector definition. Once all of that is done, we move on to calculate the breakpoint array; this is an array that defines what row each workgroup iteration will start and stop on. Once the breakpoint array is complete, we send the breakpoint array to the device with rank = 1, the total number of workgroup iterations, and the total number of rows. Finally, for each workgroup iteration, we send the corresponding device that is assigned that particular iteration, what row they start on and end on, and their designated values, and the values corresponding column position. This device will be done only after every workgroup iteration has received its data.

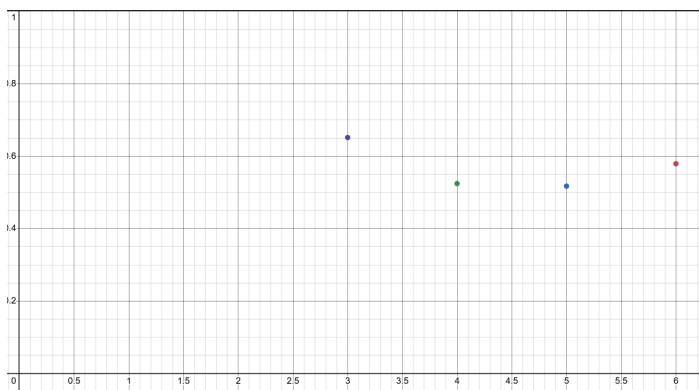
While the rank = 0 device is doing that, the devices with rank > 1 will wait to receive the necessary overhead variables. Once the particular device has received that, it will then enter a for loop that iterates through each workgroup iteration that this device is expected to compute. For a particular iteration, first, it waits to receive the starting and ending row, as well as the values and the corresponding column index to those values. Once all values have been received, it calculates each product with its corresponding x value in parallel. Once that is done, it computes the partial sum for each row this particular iteration is expected to

handle. Using the row pointer array and global index in the calculated value array, it computes all the partial sums. Once all partial sums are calculated, it sends how many partial sums it computed, followed by the computed partial sums, to the device with rank = 1. Finally, the iteration frees all allocated memory and repeats until there are no more iterations to complete.

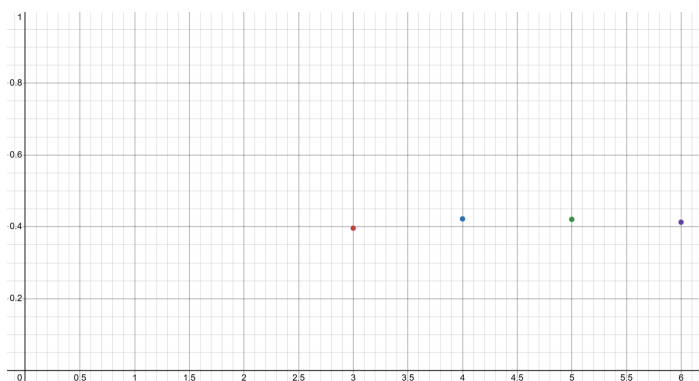
At the same time, the device with rank = 1 is first waiting for the device with rank = 0 to send the overhead variables. It then does some more overhead and enters a for loop. Each iteration of the for loop corresponds to completing one iteration of a workgroup. When the device receives the partial sums, using the breakpoint array, it sums up the partial sums with all other partial sums to create the complete product, thus finishing the algorithm.

Runtime Data:

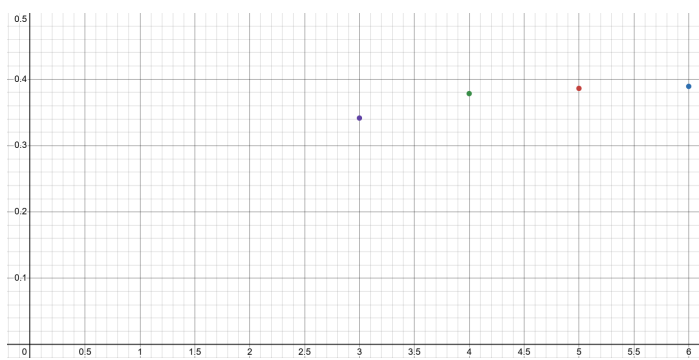
This is the runtime data for a SpMV where the number of non-zeros = 19753. The y axis is the runtime in seconds, the x axis is the number of devices.



This a graph of the runtime where number of threads per device is 3

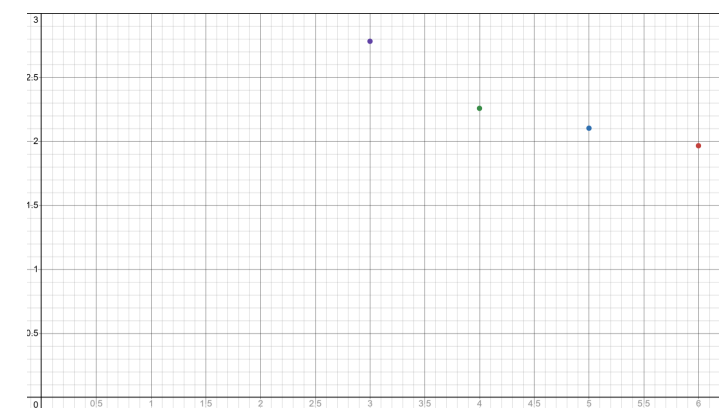


This a graph of the runtime where number of threads per device is 5

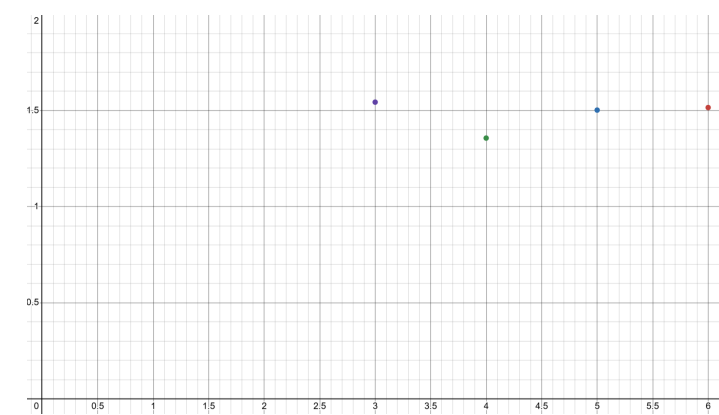


This a graph of the runtime where number of threads per device is 7

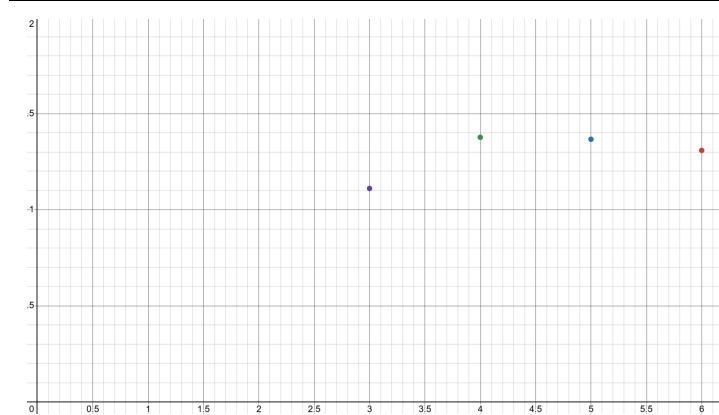
This is the runtime data for a SpMV where the number of non-zeros = 79210. The y axis is the runtime in seconds, the x axis is the number of devices.



This a graph of the runtime where number of threads per device is 3



This a graph of the runtime where number of threads per device is 5



This a graph of the runtime where number of threads per device is 7

Conclusion:

I was able to implement the SpMV algorithm on the HPC successfully. Additionally, I have seen that it works so long as the number of devices ≥ 3 and the number of threads per device ≥ 1 . Additionally, from the runtime data I gathered, we can see that this algorithm runs pretty quickly on limited resources for matrices with a NNZ $< 100,000$. Also, from that runtime data, we can see how increasing the number of threads per device consistently increases performance, whereas increasing the number of devices used does not necessarily do so. I suspect that increasing devices does not always yield performance increases because it can increase the number of communications required (this is dependent on NNZ). Additionally, it could lead to time loss due to the devices of rank = 0 and rank = 1 having to have all communication in sequence. To provide a sufficient analysis of this method, I need to develop much more rapid testing methods to test a wide variety of threads per device, the number of devices, the number of elements to compute per thread, and the number of non-zero elements.

References:

1. Genshen Chu, Yuanjie He, Lingyu Dong, Zhezha Ding, Dandan Chen, He Bai, Xuesong Wang, and Changjun Hu. 2023. Efficient Algorithm Design of Optimizing SpMV on GPU. In Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '23). Association for Computing Machinery, New York, NY, USA, 115–128.
<https://doi.org/10.1145/3588195.3593002>